

MASTER DEGREE THESIS

# An Incremental Approach to Automatic Firewall Configuration

---

**Filippo Del Ministro**

## Supervisors

prof. Fulvio Valenza

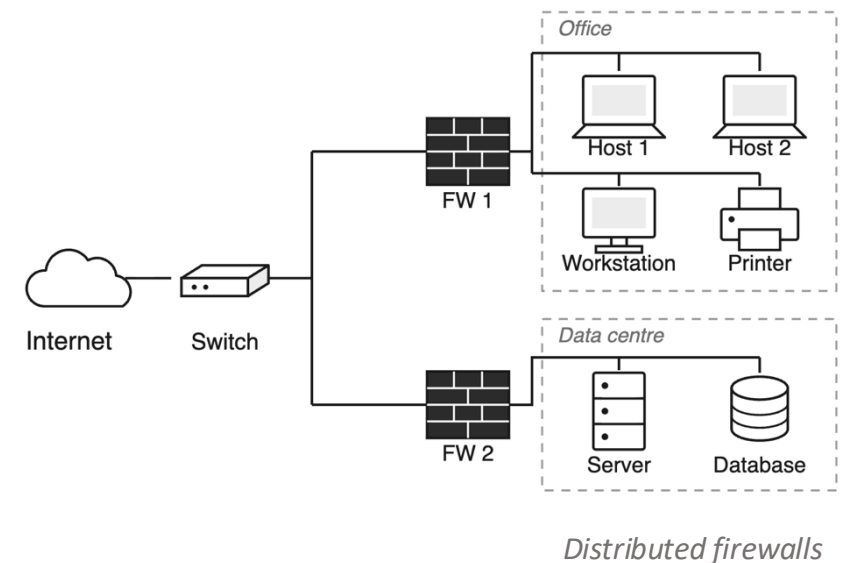
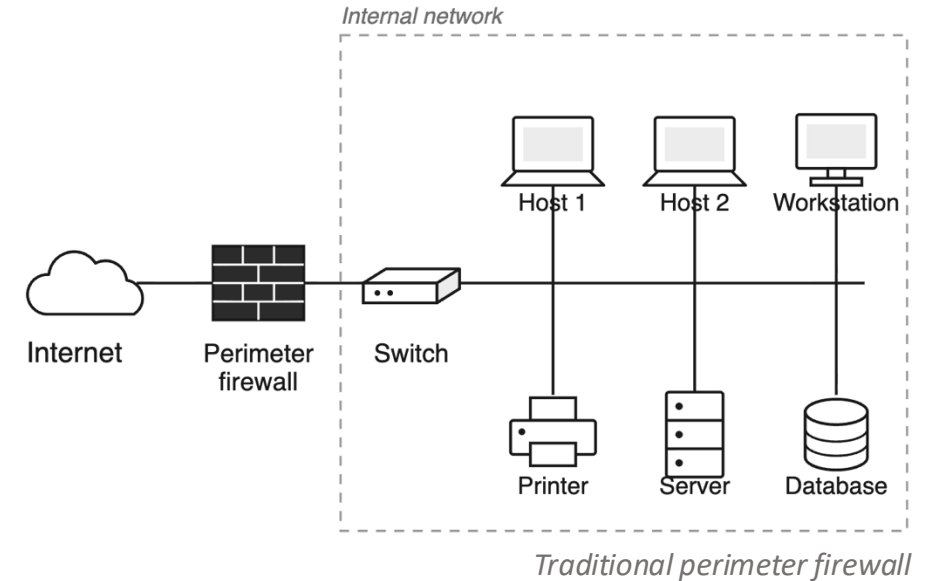
prof. Daniele Brighenti

prof. Francesco Pizzato

prof. Riccardo Sisto

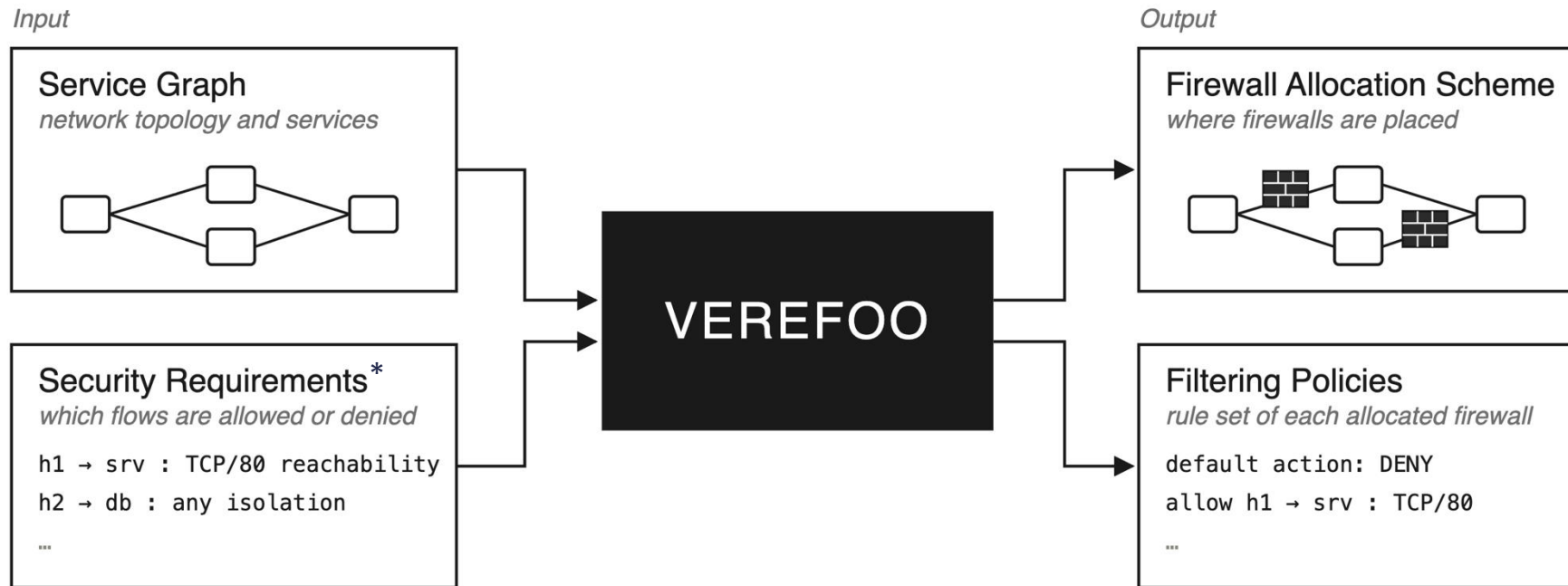
# Background

- Growing network complexity is driving a shift to distributed firewalls
- **Benefits:** finer-grained control, containment of lateral movement, tailored policies across the network
- **Trade-off:** more instances and rules to configure means more chances for conflicts; manual configuration is error-prone, with no guarantees of correctness



# What is VEREFOO?

VEREFOO is an automatic tool that allocates and configures firewalls for a given network, formally guaranteeing correctness and optimality

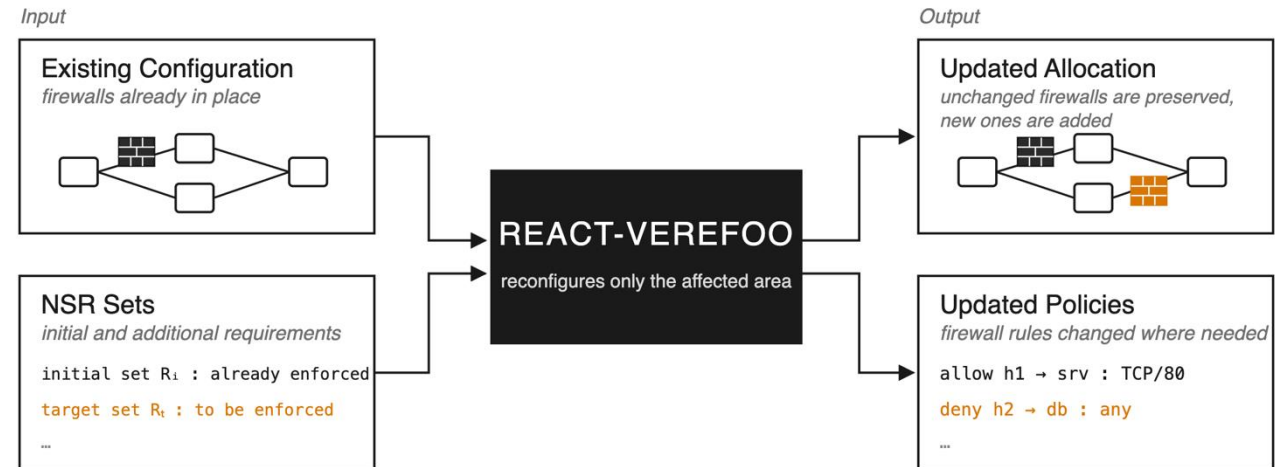


\*Network Security Requirements (NSRs) — which traffic flows must be allowed or denied

# Solution for dynamism: React-VEREFOO

React-VEREFOO automatically reconfigures an existing firewall configuration when new requirements are introduced, avoiding a full re-computation from scratch

- Input: existing configuration (firewalls already in place), initial requirements + additional requirements
- Classifies NSRs as added, kept, or deleted and **reconfigures only the affected area**
- Much faster than the monolithic version **when only a small batch of requirements changes**



## KEY IDEA

Replace the *one-shot* approach of VEREFOO with a sequence of calls to React-VEREFOO — each one adding one single requirement.

### Vanilla VEREFOO

large batch of  $N$  NSRs

▼ one expensive solver call

*speedup degrades as the batch grows*

### Iterative VEREFOO (this thesis)

1

2

3

...

$N$

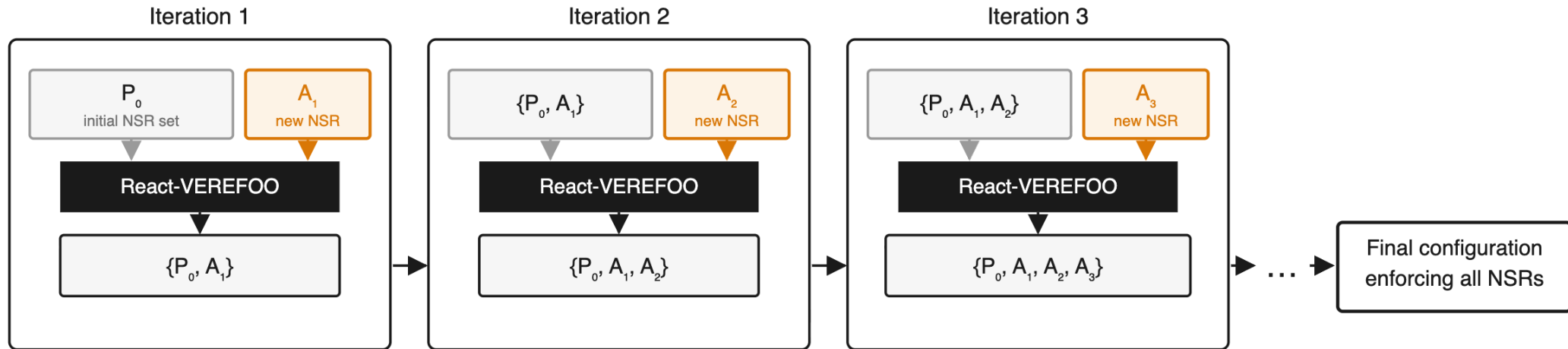
▼ many minimal solver calls

*advantage stays consistent, whatever the total*

# Thesis Contribution: Iterative VEREFOO

Pushes the incremental principle to its extreme: **introduces exactly one new NSR at a time**

- Starts from the **initial set**  $P_0$  and folds in one **additional requirement** at a time —  $A_1, A_2, A_3$ , etc.
- At each step, React-VEREFOO is invoked with everything enforced so far plus the new requirement:  $\{P_0\} + A_1$  then  $\{P_0, A_1\} + A_2$ , and so on



*Correctness is always guaranteed by React-VEREFOO; all NSRs are satisfied by the final configuration.*

# Implementation

## 1 All changes are confined to VerefooSerializer

The entry point to the VEREFOO tool

## 2 A new wrapper unpack the additional requirements

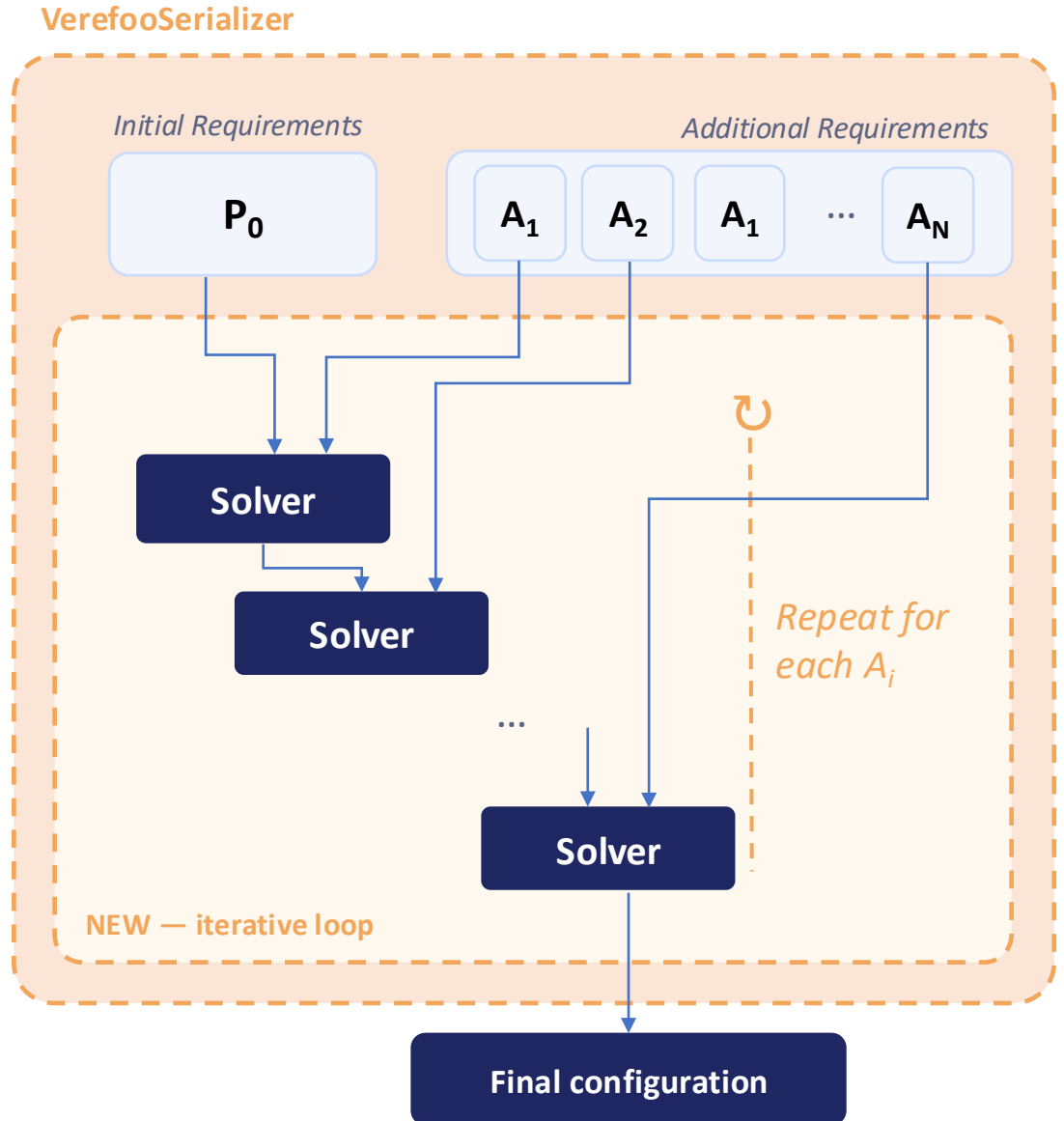
Split  $A_1, \dots, A_N$  into one call per requirement, instead of a single batch call

## 3 State updated every iteration

Current requirements and graphs are updated at each call to propagate the intermediate result into the next iteration

## 4 All other components remained untouched

MaxSMT encoding, VerefooProxy, VerefooTranslator unchanged



# Qualitative validation

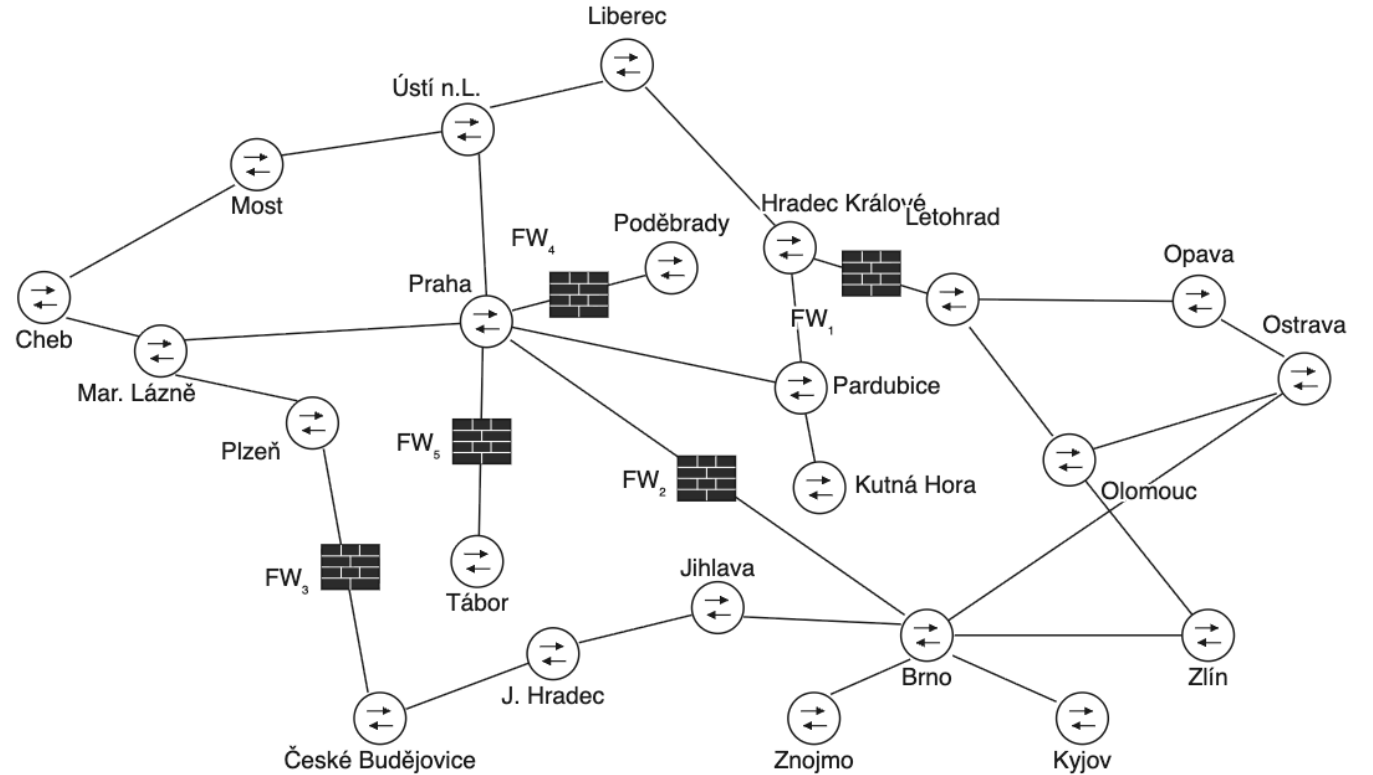
- CESNET: 23 nodes, 28 links
- $P_0$ : 10 NSRs — enforced by 3 firewalls
- 5 additional NSRs introduced *iteratively*

## Results:

**5** Firewalls  
allocated

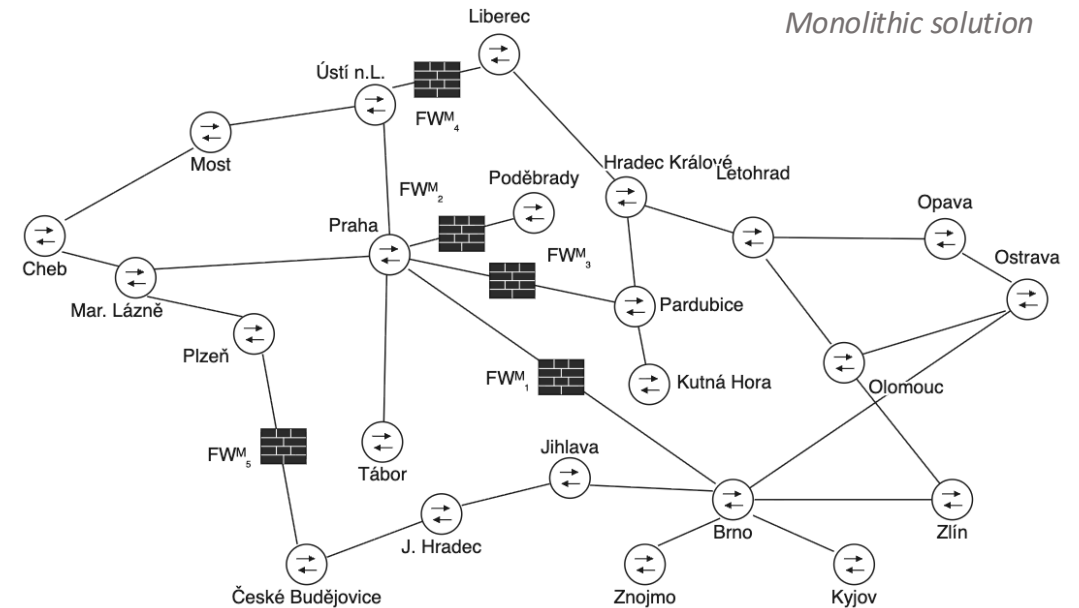
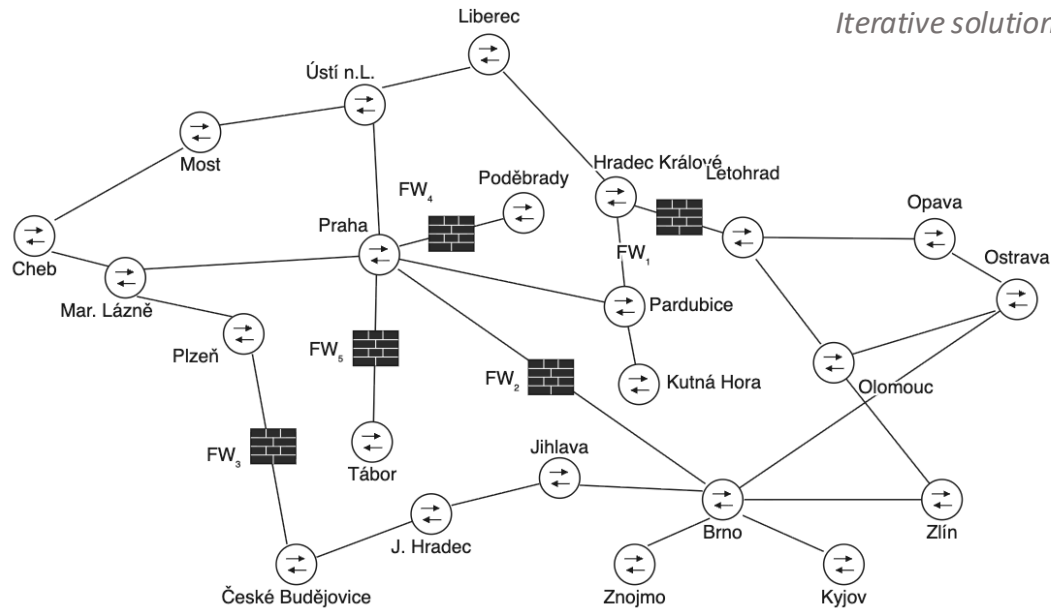
**4** Configuration  
rules

*No previously satisfied requirement  
ever broken*



# Validation: Iterative vs. Monolithic

The two results are directly **comparable**: both reach 5 firewalls, 3 of them in the exact same positions.



Iterative (left) and monolithic (right) firewall placement over the same 15 NSRs and topology

## Comparison:

- One extra rule in the iterative result; negligible cost
- **Iterative stays stable**: already valid firewall are never reconfigured

# Conclusions & Future Work

## Conclusions

- **Validated** qualitatively on a realistic CESNET topology
- **Correctness is guaranteed**, and the result is comparable to the monolithic baseline: 3 identical firewall allocations, only one extra rule
- Particularly **suited to highly dynamic environments**, where reconfiguration can become a bottleneck or a massive operational cost

## Future Work

- Quantitative benchmarking is still missing and needed: execution time against the monolithic baseline, across multiple topologies and configurations
- Further optimizations could be explored: traffic-aware NSR ordering, and support for NSR removal

**Thank you  
for your attention**



**Politecnico  
di Torino**